Build a Real-Time Cryptocurrency Tracker with Binance WebSocket API and React

Back

Mandatory

Domain

Frontend Development

Skills

API Integration | Artificial Intelligence | Data Analysis | Frontend Development | Management | Programming

Difficulty

Hard

Tools

Docker | Nginx | React | Tailwind CSS | Zustand | Binance API | CoinGecko API | Lightweight Charts

Industries

Finance

Pending Submission

Please submit your work when ready.

TIME REMAINING

1d

Deadline: **9 May 2026, 04:59 pm**

Overview Instructions Resources Submit

Report Issue

You will build a feature-rich, real-time cryptocurrency tracking dashboard. You'll learn to integrate with WebSocket APIs for live data streams, manage complex application state, implement advanced data visualizations like charts and sparklines, and persist user data using local storage. This project is highly relevant for roles in FinTech, data analytics, and any field requiring real-time, data-intensive user interfaces.

Description

Background

In the fast-paced world of financial markets, real-time data is critical. Cryptocurrency trading platforms rely on instant price updates to provide users with accurate information for making trades, managing portfolios, and monitoring market movements. Traditional REST APIs, which operate on a request-response model, are often inefficient for this use case, as they require constant polling from the client, leading to high latency and wasted resources.

This is where the WebSocket API comes in. WebSockets provide a full-duplex communication channel over a single, long-lived TCP connection, allowing servers to push data to clients in real-time. For an application like a cryptocurrency tracker, this means prices can be updated on the user's screen the moment they change on the exchange.

Building such an application requires a robust frontend architecture. You will need to manage a persistent connection to the WebSocket server, handle a continuous stream of incoming data, efficiently update the UI without performance degradation, and manage application state that includes user portfolios and price alerts. Modern state management libraries (like Zustand, Redux, or MobX) are essential for handling this complexity, while data visualization libraries (like Recharts or Lightweight Charts) are used to present complex financial data in an intuitive way.

Implementation Details

Your task is to build a single-page application that serves as a cryptocurrency dashboard. The application will fetch initial data from a REST API and then connect to a WebSocket for real-time price updates.

Step 1: Project Setup & Containerization

Initialize a new React project using your preferred toolchain (e.g., Create React App, Vite). Set up your project to be served by a lightweight


```
# docker-compose.yml
version: '3.8'
services:
  frontend:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - '8080:80' # Expose port 8080 to access the app
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost/health"] # Assumes you add a /health endpoint or file
      interval: 30s
      timeout: 10s
      retries: 3
```

Create a `.env.example` file to document environment variables. At a minimum, it should contain the WebSocket URL.

```
# .env.example
REACT_APP_BINANCE_WS_URL=wss://stream.binance.com:9443/ws
REACT_APP_COINGECKO_API_URL=https://api.coingecko.com/api/v3
```

Step 2: Fetch Initial Data

On application load, fetch a list of the top 100 cryptocurrencies by market cap from a public API like CoinGecko. This provides the initial dataset, including names, symbols, and historical data for sparklines. Display this data in a list or table view.

Step 3: Implement WebSocket Connection

Establish a connection to the Binance WebSocket API to subscribe to real-time trade streams for the displayed cryptocurrencies. You will need to handle the connection lifecycle: opening, receiving messages, handling errors, and closing the connection gracefully. Parse

incoming messages to extract the latest price for each symbol and update your application's state.

Develop the core UI components for the dashboard:

1. **Crypto List View**: A table or list showing each cryptocurrency, its current price, 24h change, and a sparkline chart of its recent performance.
2. **Search/Filter**: An input field to allow users to filter the list of cryptocurrencies by name or symbol.
3. **Portfolio Management**: A separate view or modal where users can add cryptocurrencies to their personal portfolio by specifying the coin, quantity, and average purchase price. This data must be persisted in `localStorage`.
4. **Price Alerts**: A mechanism for users to set price targets for specific coins. These alerts should also be persisted in `localStorage` and trigger a visual notification when a price target is met.
5. **Theme Switcher**: A button to toggle between light and dark themes.

Step 5: State Management and Data Persistence

Use a state management solution like Zustand to manage global state, such as the list of cryptocurrencies, portfolio data, and WebSocket connection status. Utilize `localStorage` to persist the user's portfolio and price alerts across browser sessions.

Submission Checklist

Ensure your submission repository contains the following artifacts:

- [ ] `docker-compose.yml` # Main docker compose file.
- [ ] `Dockerfile` # Dockerfile for the frontend application.
- [ ] `.env.example` # Example environment variables file.
- [ ] `src/` # Directory containing all your application source code.
- [ ] `README.md` # A detailed README with setup instructions and an overview of your project.
- [ ] `package.json` and lock file (`package-lock.json` or `yarn.lock`).

Implementation Guidelines

These are suggestions to guide your implementation, not strict requirements.

2. **State Management**: For an application with real-time updates, consider centralizing your crypto data in a global store. Zustand is lightweight and a good choice here. UI state, like the status of a modal, can often be managed locally within the component using `useState`.
3. **Performance Optimization**: Displaying a list of 100 items that update frequently can be performance-intensive. Use techniques like `React.memo` to prevent unnecessary re-renders of list items. For very large lists, consider implementing windowing or virtualization to only render the items currently in the viewport.
4. **WebSocket Management**: Wrap your WebSocket logic in a custom hook (e.g., `useWebSocket`) or a dedicated service to encapsulate connection management, subscriptions, and message handling. Implement retry logic with exponential backoff to automatically reconnect if the WebSocket connection is lost.
5. **Error Handling**: Gracefully handle potential errors, such as a failed API request for initial data or a disconnected WebSocket. Display informative messages to the user in these cases.
6. **Data Persistence**: When interacting with `localStorage`, always wrap your calls in `try...catch` blocks, as browser settings can sometimes prevent access, causing your application to crash.

FAQ

1. **Q: Do I need an API key for the Binance WebSocket?**
    1. A: No, for public market data streams like the one used in this project, you do not need an API key. You can connect directly to the public endpoint.
2. **Q: How do I subscribe to multiple streams on one WebSocket connection?**
    1. A: The Binance WebSocket API allows you to subscribe to multiple streams by sending a JSON message with the method `SUBSCRIBE` and a `params` array containing the stream names (e.g., `btcusdt@trade`, `ethusdt@trade`).
3. **Q: My component re-renders too often and the UI is slow. What can I do?**
    1. A: This is a common issue with real-time data. Ensure you are using `React.memo` on your list items. Also, check your state management logic to ensure that only the components that need the new data are re-rendering. Profiling your application with

React DevTools can help identify performance bottlenecks.

1. A: You can manually trigger this by setting a price alert very close to the current price of a volatile coin and waiting for it to be crossed. Alternatively, you can temporarily add a button in your UI that manually sets the application's state for a coin's price to a value that would trigger the alert.

5. **Q: What is the best way to structure my data in `localStorage` (ref: `localstorage-portfolio-schema`)?**

    1. A: Storing your portfolio as a JSON array of objects is a good approach. Each object can represent a holding and contain properties like `id` (e.g., 'bitcoin'), `quantity`, and `purchasePrice`. Always serialize your data with `JSON.stringify` before saving and parse it with `JSON.parse` when retrieving.

Core Requirements

1. The application must be fully containerized using Docker and Docker Compose. A `docker-compose up` command should build and start the application, making it accessible on port 8080.

    File Location: `docker-compose.yml` (repository root)

    Required Services:

    1. `frontend`: A service that builds the React application and serves it, for example, using an Nginx container.

    Required Behavior:

    1. The command `docker-compose up` must successfully start the service without any manual intervention.
    2. The application must be accessible at `http://localhost:8080`.
    3. The `frontend` service must include a functional `healthcheck`.

    Verification:

    1. Run `docker-compose up -d --build`.

2. Wait for the `frontend` service to become healthy.

2. A `.env.example` file must be present in the root of the repository, documenting all necessary environment variables for the application to run.

   File Location: `.env.example` (repository root)

   Required Content:

   1. Must contain `REACT_APP_BINANCE_WS_URL`.
   2. Must contain `REACT_APP_COINGECKO_API_URL`.
   3. Must contain placeholder values, not real secrets.

   Verification:

   1. Check for the existence of the `.env.example` file in the repository root.
   2. Parse the file and verify that the specified keys are present.

3. On initial load, the application must fetch and display a list of at least 10 cryptocurrencies.

   Required Elements:

   1. The main view must contain a list or table of cryptocurrencies.
   2. Each item in the list must have a unique `data-testid` attribute in the format `crypto-row-<symbol>`, e.g., `crypto-row-BTCUSDT`.

   Verification:

   1. Load the application at `http://localhost:8080`.

2. Use a testing tool like Playwright to check for the presence of at least 10 elements with a `data-testid` that starts with `crypto-`

---

4. The application must provide an input field that filters the list of cryptocurrencies in real-time as the user types.

Required Elements:

1. An input element with `data-testid="search-input"`.

Behavior:

1. When the application is loaded, multiple `crypto-row-*` elements are visible.
2. Typing a specific symbol (e.g., "DOGE") into the search input should filter the list so that only rows corresponding to that symbol (e.g., `crypto-row-DOGEUSDT`) remain visible.

Verification:

1. Load the application.
2. Locate the input element with `data-testid="search-input"`.
3. Type "ETH" into the input.
4. Verify that the element `crypto-row-BTCUSDT` is no longer visible.
5. Verify that the element `crypto-row-ETHUSDT` is still visible.

---

5. The application must establish a connection to the WebSocket server and expose its connection state for verification.

Required Function:

1. A function must be exposed on the `window` object: `window.getWebSocketState()`.

2. This function must return a string representing the connection state. Expected values are `"CONNECTING"`, `"OPEN"`, `"CLOSING"`,

1. Within 10 seconds of the application loading, the WebSocket connection should be established.

Verification:

1. Load the application.
2. Wait up to 10 seconds.
3. Execute `window.getWebSocketState()` in the browser console.
4. Verify that the returned value is `"OPEN"`.

---

6. The displayed price for a cryptocurrency must update in real-time based on data received from the WebSocket stream.

Required Elements:

1. For each crypto row, there must be an element displaying the price with `data-testid="price-<symbol>"`, e.g., `price-BTCUSDT`.

Behavior:

1. The text content of this element must change automatically without a page refresh as new data arrives from the WebSocket.

Verification:

1. Load the application.
2. Record the initial text content of the element with `data-testid="price-BTCUSDT"`.
3. Wait for 5 seconds to allow new messages to be received from the live WebSocket stream.
4. Read the text content of the same element again.
5. Verify that the new value is different from the initial value.

---

7. Each cryptocurrency row in the main list must display a small sparkline chart showing recent price trends.

1. Within each element identified by `data-testid="crypto-row-<symbol>"`, there must be an `svg` or `canvas` element representing the sparkline chart.
2. This chart element must have the attribute `data-testid="sparkline-<symbol>"`.

Verification:

1. Load the application.
2. Query for an element with `data-testid="crypto-row-BTCUSDT"`.
3. Within that element, verify the existence of a descendant with `data-testid="sparkline-BTCUSDT"`.

---

8. The 24-hour price change indicator must visually distinguish between positive and negative changes using a data attribute.

Required Elements:

1. An element displaying the 24h price change percentage with `data-testid="price-change-24h-<symbol>"`.

Behavior:

1. This element must have a `data-direction` attribute.
2. The value of `data-direction` must be `"up"` if the 24h change is positive or zero, and `"down"` if it is negative.

Verification:

1. Load the application.
2. Locate the element with `data-testid="price-change-24h-BTCUSDT"`.
3. Verify that it has a `data-direction` attribute.
4. Verify that the value of the attribute is either `"up"` or `"down"`.

LocalStorage Key: `cryptoPortfolio`

Schema:

1. The value must be a JSON string that parses into an array of objects.
2. Each object must contain the following keys:
    1. `id` : `string` (e.g., "bitcoin")
    2. `quantity` : `number`
    3. `purchasePrice` : `number`

Behavior:

1. When a user adds a new coin to their portfolio through the UI, an entry adhering to this schema must be added to the array in `localStorage` .

Verification:

1. Clear `localStorage` .
2. Use the UI to add a coin (e.g., 0.5 Bitcoin at a price of 50000) to the portfolio.
3. Read the `cryptoPortfolio` key from `localStorage` .
4. Parse the JSON string and verify it is an array containing an object with `id: "bitcoin"` , `quantity: 0.5` , and `purchasePrice: 50000` (or similar, based on UI interaction).

---

10. The portfolio view must correctly display the assets that are stored in `localStorage` .

Required Elements:

1. A dedicated view or section for the portfolio.
2. Each item in the portfolio display must have a `data-testid="portfolio-item-<id>"` (e.g., `portfolio-item-bitcoin`).

Behavior:

Dashboard Skill Graph Profile GPP

Verification:

1. Manually set the `cryptoPortfolio` key in `localStorage` with mock data: `[{"id":"ethereum","quantity":10,"purchasePrice":2000}]`.
2. Refresh the application and navigate to the portfolio view.
3. Verify that an element with `data-testid="portfolio-item-ethereum"` is visible.
4. Verify that an element for a non-existent portfolio item (e.g., `portfolio-item-dogecoin`) is not visible.

---

11. The portfolio view must display the calculated total value and total profit/loss based on current market prices and the user's portfolio data.

Required Elements:

1. An element with `data-testid="portfolio-total-value"`.
2. An element with `data-testid="portfolio-pl"` (Profit/Loss).

Behavior:

1. These elements must display numerical values calculated from the data in `localStorage` and the current prices from the WebSocket stream.

Verification:

1. Manually set `localStorage` with a known portfolio (e.g., 2 BTC bought at $20,000).
2. Get the current price of BTC from the UI (from `data-testid="price-BTCUSDT"`).
3. Calculate the expected total value (2 * current price) and P/L (2 * (current price - 20000)).
4. Check the text content of `data-testid="portfolio-total-value"` and verify it matches the expected total value (within a small tolerance for formatting).
5. Check the text content of `data-testid="portfolio-pl"` and verify it matches the expected P/L.

Behavior:

1. Clicking on a cryptocurrency row (e.g., `data-testid="crypto-row-BTCUSDT"`) should navigate to a new page or open a modal/overlay.
2. This detailed view must contain a larger price chart element.
3. The chart element must have `data-testid="price-chart"`.

Verification:

1. Load the application.
2. Click on the element with `data-testid="crypto-row-ETHUSDT"`.
3. Verify that an element with `data-testid="price-chart"` is now visible.

---

13. Price alerts created by the user must be persisted in `localStorage` with a specific schema.

LocalStorage Key: `cryptoAlerts`

Schema:

1. The value must be a JSON string that parses into an array of objects.
2. Each object must contain the following keys:
    1. `id`: `string` (e.g., "bitcoin")
    2. `targetPrice`: `number`
    3. `condition`: `string` (either `"above"` or `"below"`)

Verification:

1. Clear `localStorage`.

4. Parse the JSON and verify it contains an object with `id: "bitcoin"`, `targetPrice: 100000`, and `condition: "above"`.

---

14. When a cryptocurrency's price crosses a user-defined alert threshold, a visual notification must be displayed.

Required Elements:

1. A notification element that appears when an alert is triggered, with `data-testid="alert-notification-<id>"`.

Behavior:

1. Assume an alert is set for BTC to trigger if the price goes above $50,000.
2. When the price of BTC received from the WebSocket is greater than $50,000, the notification should become visible.

Verification:

1. Manually set `localStorage` with an alert for ETH to trigger above a price that is slightly higher than the current price (e.g., current price + $10).
2. Wait for the live WebSocket feed to update the price of ETH and cross the threshold.
3. Verify that the element `data-testid="alert-notification-ethereum"` becomes visible within 10 seconds of the price crossing.

---

15. The application must support toggling between a light and dark theme, and the current theme state must be reflected on the main HTML element.

Required Elements:

1. A button or toggle switch with `data-testid="theme-switcher"`.

Behavior:

Verification:

1. Load the application.
2. Check if the `<html>` element has the class `dark`. Let's assume it does not by default.
3. Click the element with `data-testid="theme-switcher"`.
4. Verify that the `<html>` element now has the class `dark`.
5. Click the button again.
6. Verify that the `<html>` element no longer has the class `dark`.